Forecasting Bike Rental Demand

Machine Learning Mini Project — Project No. 87
Based on the Stanford CS229 reference work by Jimmy Du, Rolland He, and Zhivko Zhechev

1. Problem Statement

The objective of this project is to predict the number of bike rentals in a given hour using temporal, calendar, and weather-related information. Accurate demand forecasting helps bike-sharing operators anticipate usage patterns, improve bike availability, and support efficient redistribution. The project builds on the same bike-sharing demand forecasting problem and underlying dataset used in the Stanford CS229 reference work, while implementing a more time-aware validation strategy and a deployable prediction demo.

2. Dataset Details

We used the UCI Bike Sharing hourly dataset, which contains **17,379 hourly observations** collected during **2011–2012**. Each row represents one hour and includes variables such as season, holiday status, working-day status, weather situation, temperature, feels-like temperature (*atemp*), humidity, windspeed, and the total number of rentals (*cnt*).

A key data-design rule was leakage prevention. Since **cnt = casual + registered**, the columns **casual** and **registered** were excluded from the predictor set. The identifier column **instant** was also removed because it is not a meaningful predictive feature.

3. Data Understanding and Feature Engineering

Exploratory analysis was first performed using **data_understanding.py**. This stage checked data types, missing values, duplicate hourly records using the pair *(dteday, hr)*, date range coverage, target distribution, and relationships between demand and weather variables. The strongest patterns were temporal: bike rentals showed clear hourly peaks, and working days exhibited commuter-style morning and evening demand spikes.

Feature engineering was then implemented in **feature_engineering.py**. Cyclic sine/cosine encodings were created for hour, weekday, and month so that cyclical continuity is preserved (for example, 23:00 remains close to 00:00). Additional derived features included **is_weekend**, **is_rush_hour**, and **peak_time**, which distinguish weekday and non-working-day peak periods. The final feature set combined numerical weather variables, temporal encodings, and categorical calendar/weather variables.

4. Modeling Approach

The data was sorted chronologically and split into an **80/20 temporal holdout**: the earlier 80% formed the training period and the later 20% formed the final test period. Within the training period, model selection used **TimeSeriesSplit(4)** so that validation observations always occur after their corresponding training observations. This avoids using future information to validate earlier periods and makes the evaluation closer to a real forecasting setup.

Four regression models were compared: **Linear Regression**, **K-Nearest Neighbors (KNN) Regression**, **Random Forest Regression**, and **Gradient Boosting Regression**. All models were implemented through sklearn pipelines defined in **model_training.py**. Numerical features were standardized for Linear Regression and KNN, while tree-based models used numerical passthrough. Categorical features were one-hot encoded for all models.

Hyperparameters were tuned using **GridSearchCV** with **RMSLE** as the primary scoring metric. RMSLE was chosen because it is appropriate for count-style demand prediction and focuses on proportional prediction error. The final experiment orchestration was handled in **train_models.py**, and metrics/plots were produced using **evaluation.py**.

5. Brief Implementation Overview

File	Role
data_understanding.py	EDA, quality checks, and generation of exploratory plots
feature_engineering.py	Feature creation, cyclic encodings, and temporal train/test split
model_config.py	Central definitions for features, constants, paths, and leakage columns
model_training.py	Preprocessing pipelines, TimeSeriesSplit, and model search definitions
train_models.py	End-to-end experiment runner, model comparison, and final model selection
evaluation.py	Metric functions and final comparison/diagnostic plots
demo_app.py	Streamlit interface for interactive prediction using the saved model

6. Results

Model	CV RMSLE	Test RMSLE	RMSE	MAE	R^2
Random Forest	0.4833	0.4115	80.43	53.13	0.8667
KNN Regression	0.5976	0.5335	122.35	83.59	0.6914
Gradient Boosting	0.7257	0.6278	96.87	68.24	0.8066
Linear Regression	1.2964	0.9813	112.48	83.87	0.7392

Random Forest achieved the best performance among the four models. Importantly, it was selected using the **lowest cross-validation RMSLE** rather than the holdout test score, preserving the independence of the test period. The best Random Forest configuration used **400 trees**, **no fixed maximum depth**, and **min_samples_leaf = 1**.

7. Conclusions

- Random Forest was the strongest model among those evaluated, indicating that hourly bike-rental demand contains nonlinear patterns and feature interactions that tree-based ensembles capture effectively.
- Chronological holdout evaluation and TimeSeriesSplit provided a more realistic forecasting framework than ordinary random cross-validation and directly addressed the temporal-evaluation limitation noted in the reference paper.
- Cyclic time encoding and behavior-oriented features such as is_rush_hour and peak_time improved how temporal structure was represented in the feature set.
- The trained production model was integrated into a Streamlit-based demo, where user inputs are converted through the same feature-engineering logic used during training, keeping training and inference consistent.

Overall, the project demonstrates a complete machine learning workflow: data understanding, feature engineering, time-aware model selection, comparative evaluation, and interactive deployment for practical hourly bike-demand prediction.